CampusStay: 8-10 minute demonstration video guide

This is a recording plan, not a completed student video. Your team must record its own live Windows execution and each member's own explanation. Use the institution's required format and sharing location. Aim for approximately 9:30; the installation guide itself is not the video script.

Before the recording

Complete a successful fresh-clone setup and run_all on the machine you will record. Fill actual team details and contribution entries. Practise explaining the mechanisms without reading the report word for word.

Have these ready: Terminal A with scripts/run.py, Terminal B in the same project root, the UI at http://127.0.0.1:8180/, the architecture diagram/report, Git history and the three challenge evidence folders. Keep the database server running. Use readable terminal/browser text. The payment and notification providers remain local mocks.

Recording timeline and speaking cues

0:00-0:50 - team and problem

Show the project title and actual members. Each member introduces their real responsibility. Explain that CampusStay is a synthetic campus-guesthouse example with one room for one night per booking. State the contended resource: availability for a room type/date. Name the unsafe operation: creating a booking/payment twice.

Adapt these speaking points to your own understanding: We needed separate business capabilities rather than technical-layer services. The visitor makes one request, but correct completion spans inventory, payment and notification. The application demonstrates what happens when requests race, repeat or fail partway through.

0:50-2:20 - service boundaries and data

Show the component/data ownership diagram. Explain Python Inventory and Reservation, PHP Payment and Notification. Each has its own folder, dependency file, .env and database login. The optional UI is a client and has no database credentials.

Show one /health response and one /openapi.json. Explain REST/JSON and correlation IDs. Explain that keeping a peer booking/payment ID does not mean a cross-database foreign key. All foreign keys stay within one service's database.

Describe the main trade-off: REST makes cross-language communication simple, while distributed completion requires explicit state and recovery. A single shared SQL transaction cannot span these independently owned service databases in this design.

2:20-4:20 - successful flow and notification retry

Use the UI to create five rooms on a future date. Select notification Fail once, then retry. Book a room and show CONFIRMED, notification SENT and availability four of five. Show the recorded booking steps and inspect the API result.

In Terminal B run the scripted transaction so the recording includes a real terminal/API call:

```
Get-Date -Format o
.\.venv\Scripts\python.exe scripts\e2e.py
```

Explain the price comes from Inventory in integer minor units. Reservation only confirms after a successful charge. A notification failure does not undo a paid confirmed booking; its retry updates the same notification record.

4:20-5:20 - declined payment

Select Declined payment and create a new booking. Show CANCELLED and restored availability. Explain PENDING, RESERVED, PAYMENT_PENDING, COMPENSATING, CANCELLED. A decline is a known failure, so releasing the hold is safe. An unknown payment result is different: keep the hold until the outcome is resolved.

5:20-6:25 - Challenge 1

Run the challenge, with Terminal A still serving the application:

```
Get-Date -Format o
.\.venv\Scripts\python.exe scripts\challenge_concurrency.py `
  --requests 20 --capacity 5
```

Point out exactly five successes and fifteen rejections. Open the matching database_state.json and show final stock/holds. Explain the conditional UPDATE with available > 0 and the InnoDB transaction. Two callers doing a plain SELECT followed by an UPDATE could both act on stale availability.

6:25-7:30 - Challenge 2

```
Get-Date -Format o
.\.venv\Scripts\python.exe scripts\challenge_idempotency.py
```

Show sequential replays, a changed-payload 409 and 20 simultaneous identical first requests. Explain that the request key is stored with a semantic fingerprint and response. The local effect and response commit together. Stable downstream keys and unique booking IDs prevent duplicate effects. Point to one booking, hold, payment and notification in the evidence.

7:30-8:35 - Challenge 3C

```
Get-Date -Format o
.\.venv\Scripts\python.exe scripts\challenge_saga.py
```

Show the forced step-3 decline, compensation, room restoration and repeated compensation check. Explain why this is an explicit saga rather than a database rollback across services. If a successful payment existed before compensation, refund is required before releasing its room.

8:35-9:30 - tests, Git, limits and disclosure

Show your actual unit-test output, run_summary.json and a meaningful contribution commit from each member. The full suite is run with the normal launcher stopped; you may show its existing genuine results here rather than consuming the video on a full rerun. If you run it live, stop Terminal A first.

```
git log --oneline --all -8
```

Explain at least one limitation: one-night scope, local mocks, no user authentication, or manual resolution for permanently uncertain payment. State the true AI assistance and the real work your team reviewed, changed and validated. Do not claim you independently authored generated starter code.

Suggested member handoffs

Responsibility	Suggested speaking section
Inventory owner	Capacity rule and concurrent-request evidence
Reservation owner	State machine, orchestration, compensation and recovery
Payment owner	Integer amounts, declined/slow mocks, idempotency and refunds
Notification owner	Confirmed-state rule, failed-first-attempt and safe retry

Use your actual team size and work. A member may own multiple areas. Everyone must speak and be able to explain the overall architecture, not only their file names.

What screenshots and evidence must show

If including terminal screenshots, show the command, output and timestamp. Keep the original text/JSON logs in evidence. A browser image alone does not prove 20 concurrent requests or database integrity. The challenge scripts and database snapshots provide that proof.

Review the complete recording

Watch the entire file before upload. Confirm 8-10 minutes, audible members, readable terminal output, both success/failure, all selected challenges and no exposed configuration passwords. Verify that the repository/video access works for faculty. Add the actual video/Drive links to the report and rebuild it after entering the final details.